

E1 251 Course Project

Reconstruction from Non-Uniform Samples Using a DCT- l_p Prior(MM-ADMM)

Dwaipayan Haldar(SR. No.: 27128)

1 Description of Experimental setup

1.1 Model Images

Images used for the project are `cameraman.tif` and `lena`. Both have a resolution of 256×256 and normalized to $[0, 1]$ using `img_as_float` function of `skimage` package.

1.2 Sampling and Noise

I have used binary random masks using the `numpy.random` with $r \in \{0.2, 0.3, 0.5\}$, where r is the fraction of pixels used.

Noise $\eta_i \sim \mathcal{N}(0, \sigma^2)$, where σ corresponds to an SNR of 30dB. A closed form value of σ can be derived as follows:

$$\text{SNR} = 20 \log_{10} \frac{\|Wx^*\|_2}{\|\eta\|_2} = 30 \implies \frac{\|Wx^*\|_2}{\|\eta\|_2} = 10^{1.5} \implies \|\eta\|_2^2 = \frac{\|Wx^*\|_2^2}{10^3}$$

$\therefore \mathbb{E}(\eta_i^2) = \sigma^2$ (Normal Distribution with 0 Mean)

$$\implies \|\eta\|_2^2 = N\sigma^2 \implies \sigma^2 = \frac{\|Wx^*\|_2^2}{M \cdot 10^3} \implies \sigma = \sqrt{\frac{\|Wx^*\|_2^2}{M \cdot 10^3}}$$

This is the formula used to calculate the random noise in A.1.1.

1.3 Choice of parameters, Evaluation and Experiments

Experiments are run for each choice of (r, p) . Results are shown in 2 for best λ in case of each (r, p) . Best λ is chosen on the basis of the highest PSNR value. Table.1 gives the concise report for both the images.

2 Results

2.1 Cameraman

Fig.1 gives the original image, sampled noisy image and the Reconstructed image for sampling percentage 0.2. Similarly, Fig.2 and Fig.3 gives the same results for sampling percentage 0.3 and 0.5 respectively. Fig.4 gives the PSNR vs λ plot for $r = 0.5$ and $p \in \{0.5, 0.7, 0.9\}$. Fig.5 gives the Objective Function vs iteration step, No. of ADMM iteration vs iteration step, Relative error vs iteration step plots for $r = 0.5$, $p = 0.3$ and $\lambda = 0.01$ which is the best λ for the given configuration. Qualitatively, $p = 0.5$ outperforms $p = 0.4, 0.3$ which would be evident from Table.1, also for $r = 0.5$ we get better results than $r=0.2, 0.3$ which is quite expected.

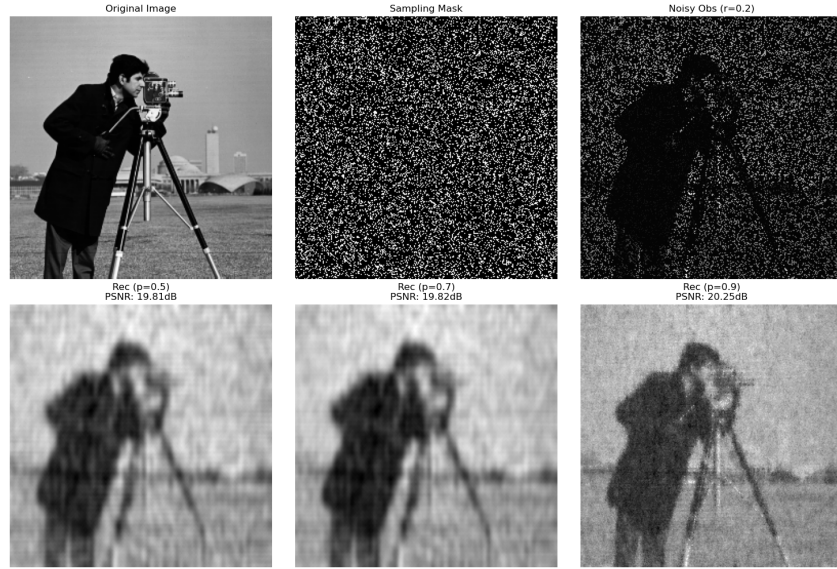


Figure 1: (First Row)Original Image, Sampling Mask, Noisy Masked Image (Second Row)Reconstructed Image($p = 0.5, 0.7, 0.9$) for $r = 0.2$

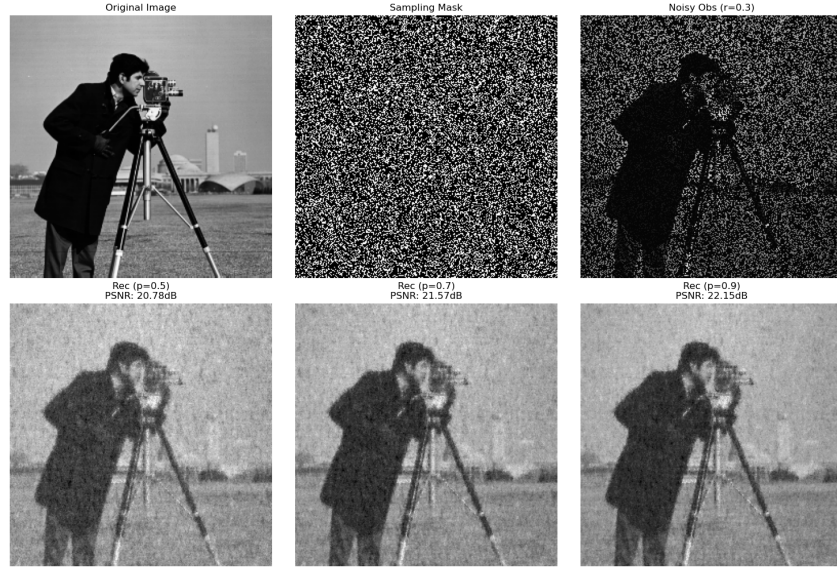


Figure 2: (First Row)Original Image, Sampling Mask, Noisy Masked Image (Second Row)Reconstructed Image($p = 0.5, 0.7, 0.9$) for $\mathbf{r} = \mathbf{0.3}$

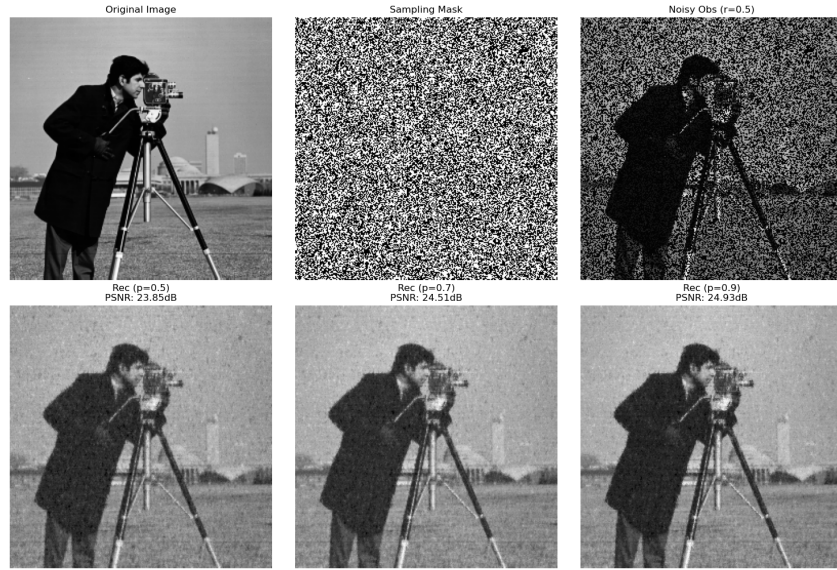


Figure 3: (First Row)Original Image, Sampling Mask, Noisy Masked Image (Second Row)Reconstructed Image($p = 0.5, 0.7, 0.9$) for $\mathbf{r} = \mathbf{0.5}$

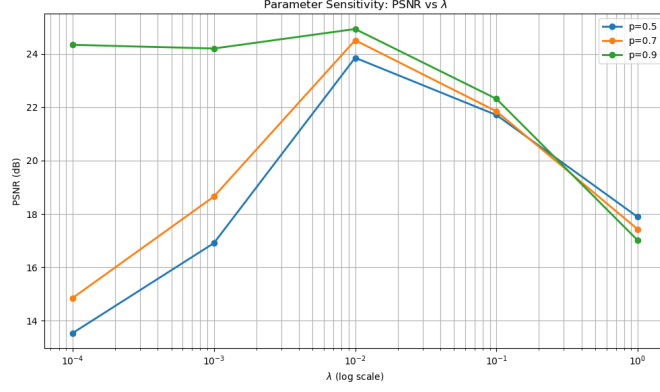


Figure 4: PSNR vs λ for 3 combination of (r,p) where $r = 0.5$ and $p \in \{0.5, 0.7, 0.9\}$

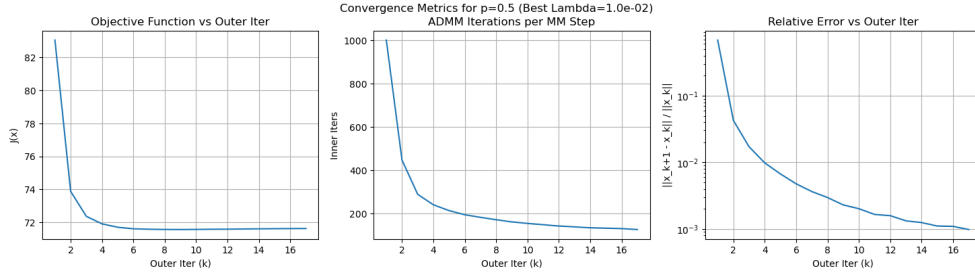


Figure 5: (From left) Objective Function vs iteration step, No. of ADMM iteration vs iteration step, Relative error vs iteration step for $(r,p,\lambda(r,p))=(0.5, 0.3, 0.01)$

2.2 Lena

Fig.6 gives the original image, sampled noisy image and the Reconstructed image for sampling percentage 0.2. Similarly, Fig.7 and Fig.8 gives the same results for sampling percentage 0.3 and 0.5 respectively. Fig.9 gives the PSNR vs λ plot for $r = 0.5$ and $p \in \{0.5, 0.7, 0.9\}$. Fig.10 gives the Objective Function vs iteration step, No. of ADMM iteration vs iteration step, Relative error vs iteration step plots for $r = 0.5$, $p = 0.3$ and $\lambda = 0.01$ which is the best λ for the given configuration. Similar results are seen for lena image also. Qualitatively, $p = 0.5$ outperforms $p = 0.4, 0.3$ which is also supported from Table.1, also for $r = 0.5$ we get better results than $r=0.2, 0.3$ which is quite expected.

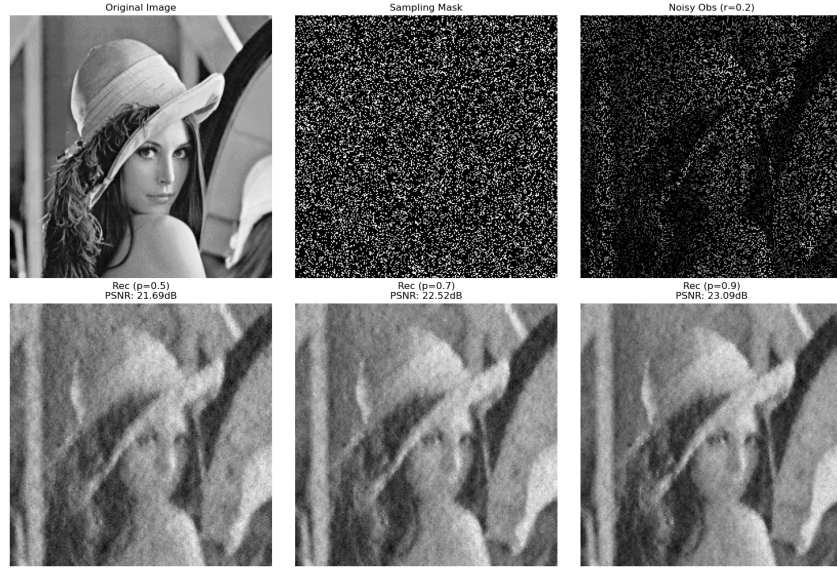


Figure 6: (First Row)Original Image, Sampling Mask, Noisy Masked Image (Second Row)Reconstructed Image($p = 0.5, 0.7, 0.9$) for $\mathbf{r} = \mathbf{0.2}$

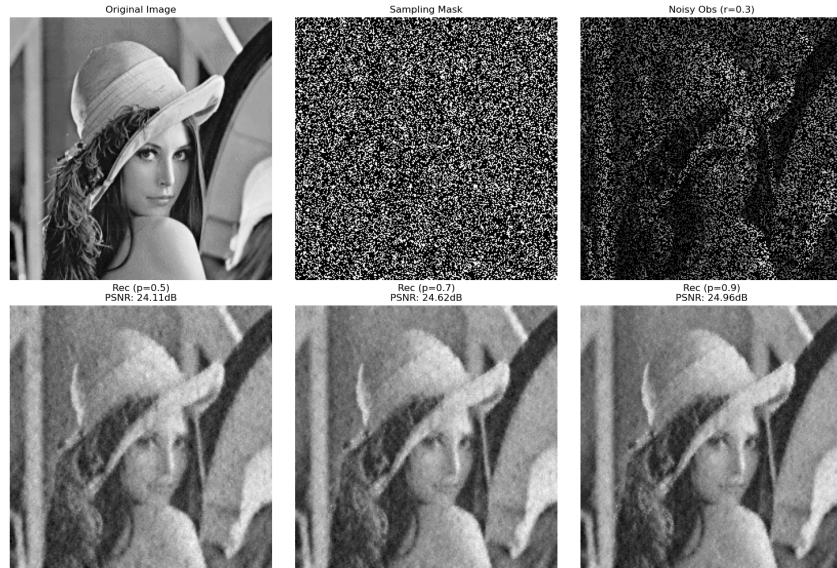


Figure 7: (First Row)Original Image, Sampling Mask, Noisy Masked Image (Second Row)Reconstructed Image($p = 0.5, 0.7, 0.9$) for $\mathbf{r} = \mathbf{0.3}$

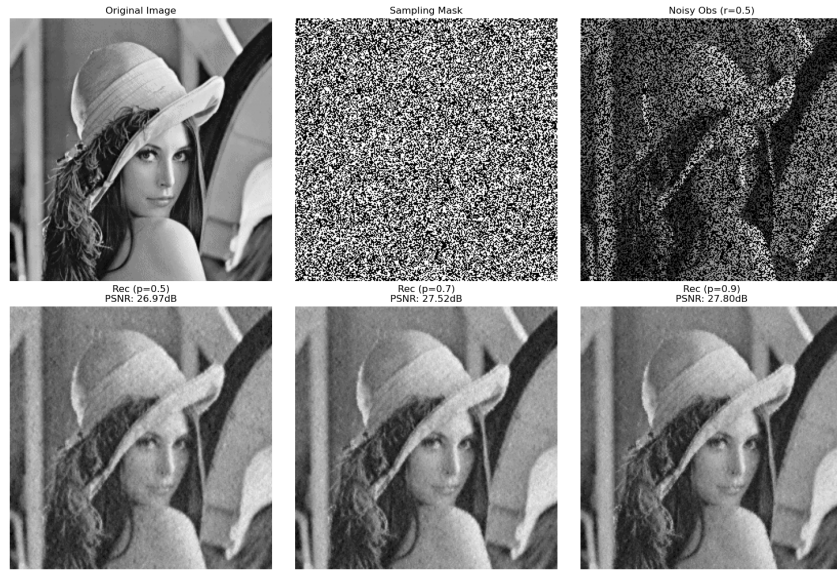


Figure 8: (First Row)Original Image, Sampling Mask, Noisy Masked Image (Second Row)Reconstructed Image($p = 0.5, 0.7, 0.9$) for $r = 0.5$

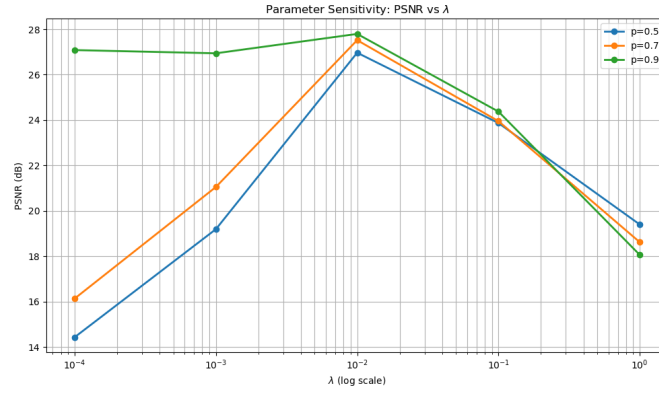


Figure 9: PSNR vs λ for 3 combination of (r, p) where $r = 0.5$ and $p \in \{0.5, 0.7, 0.9\}$

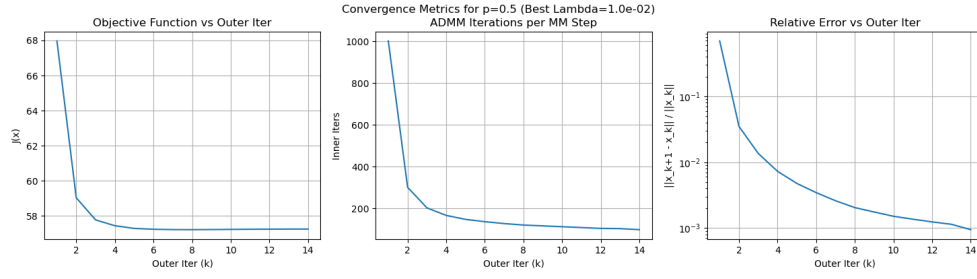


Figure 10: (From left) Objective Function vs iteration step, No. of ADMM iteration vs iteration step, Relative error vs iteration step for $(r,p,\lambda(r,p))=(0.5, 0.3, 0.01)$

2.3 Combined Results

Table 1: Comparison of (r,p) values for both the images

Image	r	p	$\lambda(r,p)$	PSNR	Relative Error	Runtime(s)
cameraman	0.2	0.5	0.1	19.81	0.194	1.92
		0.7	0.1	19.82	0.194	1.73
		0.9	0.01	20.25	0.184	11.71
	0.3	0.5	0.01	20.78	0.173	6.57
		0.7	0.01	21.57	0.158	6.54
		0.9	0.01	22.15	0.148	10.90
	0.5	0.5	0.01	23.85	0.122	7.32
		0.7	0.01	24.51	0.113	3.8
		0.9	0.01	24.93	0.107	4.31
lena	0.2	0.5	0.01	21.69	0.159	7.12
		0.7	0.01	22.52	0.144	6.72
		0.9	0.01	23.09	0.135	6.25
	0.3	0.5	0.01	24.11	0.120	4.55
		0.7	0.01	24.62	0.113	7.25
		0.9	0.01	24.96	0.109	7.64
	0.5	0.5	0.01	26.97	0.086	2.67
		0.7	0.01	27.52	0.081	2.61
		0.9	0.01	27.80	0.078	3.04

3 Discussion

3.1 Effect of λ

λ is the weight given to the sparsity term. Here λ is varied over the log scale $\{10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$. Almost in all the cases, $\lambda = 0.1$ or 0.01 is the winner. That is expected since $\lambda = 10^{-4}$ means too less weightage to the sparsity term. So, the images are likely to deviate from natural images. Because, natural images is sparse in the DCT domain. $\lambda = 1$ is giving too much weightage to the sparsity term. So, the image is likely to be oversmoothed. So, a moderate value of λ is expected to give a balanced approach.

3.2 Effect of p

p is another parameter that controls the sparsity. As per Table.1, it is quite evident that $p = 0.9$ comes to be the winner in all the cases. This is because $p = 0.5$ is an over aggressive norm for the sparsity term. Whereas $p = 0.9$ gives the most effective balance for the sparsity prior.

3.3 Effect of r

Understanding the effect of r is pretty trivial. As per Table.1, with increase in r there is a consistent increase in PSNR, consistent decrease in relative error. r is the fraction of samples selected. It is evident that if more samples are selected the final results will come out to better than less no. of samples selected.

The next section provides the commented code for all the functions used. All the output images are attached in the zip file. It contains all the plots for all possible combinations. `src.py` contains the source code for all the functions. `wrapper.ipynb` contains the outputs of those functions.

A Code

A.1 Main Functions

This section contains all the main functions used for the project namely the random sampling function, the MM inner loop, the MM-ADMM complete loop.

A.1.1 Sampling Function

```
1 def sampling(img, r):
2     """
3     Argument: img and sampling ratio r
4     Output: Sampled Noisy Vector m and indices idx
5     """
6     img_vector = img.flatten()
7     N = img_vector.shape[0]
8     M = int(np.round(r * N))
9
10    idx = np.random.choice(N, size=M, replace=False)
11    Wx_ = img_vector[idx]
12
13    # Noise addition
14    Wx_l2_2 = (np.linalg.norm(Wx_))**2
15    sigma = np.sqrt(Wx_l2_2 / (1000 * M)) # Approx 30dB SNR
16    noise = np.random.normal(0, sigma, (M,))
17    return (Wx_ + noise), idx
```

A.1.2 ADMM Inner loop

```
1 def admm_inner_solver(N, m, idx, w_k, x_init, y_prev, lammbda, rho
2     , img_shape, max_admm_iter=1000, tol=1e-4):
3     """
4     Solves the Weighted L1 subproblem using ADMM.
5     """
6     x = x_init.copy()
7     v = y_prev.copy()
8     u = np.zeros_like(v)
9
10    mask = np.zeros(N)
11    mask[idx] = 1.0
12
13    m_full = np.zeros(N)
14    m_full[idx] = m
15
16    iteration = 0
```

```

17     for t in range(max_admm_iter):
18         v_prev = v.copy()
19
20         # x-update
21         s_t = idct2(v - u, img_shape)
22         x = mask * ((m_full + rho * s_t) / (1 + rho)) + (1 - mask)
23         * s_t
24
25         # v-update (DCT Domain)
26         dct_x = dct2(x, img_shape)
27         z_t = dct_x + u
28
29         # Weighted Soft Thresholding
30         threshold = (lammbda / rho) * w_k
31         v = soft_threshold(z_t, threshold)
32
33         # u-update (Dual)
34         u = u + dct_x - v
35
36         # Convergence Check
37         r_norm = np.linalg.norm(dct_x - v) # Primal residual
38         s_norm = np.linalg.norm(rho * (v - v_prev)) # Dual
39         residual
40
41         iteration = t + 1
42         if r_norm < tol and s_norm < tol:
43             break
44
45     return x, v, iteration

```

A.1.3 MM ADMM Outer loop

```

1 def mm_admm(N, m, idx, rho, lammbda, p, img_shape, max_mm_iter
2     =100, tol=1e-3, epsilon=1e-6):
3     # Initialize x with zero-filled observations
4     x = np.zeros(N)
5     x[idx] = m
6
7     # Initial DCT coefficients
8     v = dct2(x, img_shape)
9
10    list_iteration_admm = []
11    list_objective_function = []
12    list_relative_error = []
13
14    for k in range(max_mm_iter):
15        x_prev = x.copy()
16        y_prev = v

```

```

16
17     # MM Weight Update
18     w_k = p * (np.abs(y_prev) + epsilon)**(p - 1)
19
20     # ADMM Inner Solver
21     x, v, inner_iters = admm_inner_solver(
22         N, m, idx, w_k, x_prev, y_prev, lammbda, rho,
img_shape
23     )
24
25     # Convergence Check
26     norm_x = np.linalg.norm(x)
27     rel_error = np.linalg.norm(x - x_prev) / max(1, norm_x)
28
29     # Logging
30     obj_val = objective_function(idx, x, m, lammbda, p,
img_shape)
31     list_objective_function.append(obj_val)
32     list_iteration_admm.append(inner_iters)
33     list_relative_error.append(rel_error)
34
35     if rel_error < tol:
36         break
37
38     return x, list_objective_function, list_iteration_admm,
list_relative_error

```

A.2 Auxiliary Functions

A.2.1 Plotting Function

This is the outermost function, when we call with image and r value, gives all the plots and all the values like PSNR, Relative Error etc. It is the function that is called for output in jupyter notebook.

```

1 def reconstruct(img, r):
2     img_shape = img.shape
3     N = img_shape[0] * img_shape[1]
4
5     # Generate Samples
6     m, idx = sampling(img, r)
7
8     # For Visualization
9     sampling_mask_image = sampling_mask(N, idx).reshape(img_shape)
10    noisy_image = Wtranspose(N, m, idx).reshape(img_shape)
11
12    # Storage for results across different p values

```

```

13     results_by_p = [] # Will store dictionary of results for each
14     p
15
16     p_values = [0.5, 0.7, 0.9] # As per Algorithm 1
17
18     print(f"Starting Reconstruction for r={r}...")
19     start_time_total = time.time()
20
21     for p in p_values:
22         print(f"\n Processing p={p} ")
23
24         # Initialize lists to store sensitivity data for this p
25         psnrs_for_plot = []
26         lambdas_for_plot = []
27
28         best_metrics = {
29             "psnr": -np.inf,
30             "lambda": None,
31             "image": None,
32             "obj_hist": [],
33             "iter_hist": [],
34             "err_hist": []
35         }
36
37         # Grid search for Lambda
38         lambda_grid = np.logspace(-4, 0, num=5)
39
40         for l in lambda_grid:
41             start_time_lambda = time.time()
42
43             # Run the algorithm
44             x_rec_flat, obj_hist, iter_hist, err_hist = mm_admm(
45                 N, m, idx, rho=4.0, lammbda=l, p=p, img_shape=
46                 img_shape
47             )
48
49             end_time_lambda = time.time()
50             runtime_lambda = end_time_lambda - start_time_lambda
51
52             # Reshape and compute metrics
53             x_rec = x_rec_flat.reshape(img_shape)
54             curr_psnr = psnr(x_rec, img)
55             curr_rel_error = relative_change(x_rec, img)
56
57             # Store data for plotting
58             psnrs_for_plot.append(curr_psnr)
59             lambdas_for_plot.append(l)
60
61             # Update best metrics if this is the best PSNR so far

```

```

60         if curr_psnr > best_metrics["psnr"]:
61             best_metrics["psnr"] = curr_psnr
62             best_metrics["lambda"] = l
63             best_metrics["image"] = x_rec
64             best_metrics["obj_hist"] = obj_hist
65             best_metrics["iter_hist"] = iter_hist
66             best_metrics["err_hist"] = err_hist
67             best_metrics["rel_error"] = curr_rel_error
68             best_metrics["runtime"] = runtime_lambda
69
70         # Save the sensitivity lists into the dictionary
71         best_metrics["psnr_list_for_plot"] = psnrs_for_plot
72         best_metrics["lambda_list_for_plot"] = lambdas_for_plot
73
74         results_by_p.append(best_metrics)
75         print(f"Best for p={p}: Lambda={best_metrics['lambda']:.1e
76 }, PSNR={best_metrics['psnr']:.2f}")
77         print(f"Runtime for best lambda: {best_metrics['runtime']:.4f}s,
78 Relative error: {best_metrics['rel_error']:.4e}")
79
80     end_time_total = time.time()
81     print(f"\nTotal runtime: {end_time_total - start_time_total:.2f}s")
82
83     # Plotting
84
85     # 1. Visual Comparison
86     plt.figure(figsize=(15, 10))
87
88     plt.subplot(2, 3, 1)
89     plt.title("Original Image")
90     plt.imshow(img, cmap="gray")
91     plt.axis('off')
92
93     plt.subplot(2, 3, 2)
94     plt.title("Sampling Mask")
95     plt.imshow(sampling_mask_image, cmap="gray")
96     plt.axis('off')
97
98     plt.subplot(2, 3, 3)
99     plt.title(f"Noisy Obs (r={r})")
100     plt.imshow(noisy_image, cmap="gray")
101     plt.axis('off')
102
103     for i, p in enumerate(p_values):
104         plt.subplot(2, 3, 4 + i)
105         plt.title(f"Rec (p={p})\nPSNR: {results_by_p[i]['psnr']:.2f}dB")

```

```

105     plt.imshow(results_by_p[i]['image'], cmap="gray")
106     plt.axis('off')
107 plt.tight_layout()
108 plt.show()
109
110 # 2. Convergence Metrics
111 for i, p in enumerate(p_values):
112     res = results_by_p[i]
113     k_range = range(1, len(res['obj_hist']) + 1)
114
115     plt.figure(figsize=(18, 4))
116     plt.suptitle(f"Convergence Metrics for p={p} (Best Lambda
117 ={res['lambda']:.1e})")
118
119     plt.subplot(1, 3, 1)
120     plt.plot(k_range, res['obj_hist'])
121     plt.title("Objective Function vs Outer Iter")
122     plt.xlabel("Outer Iter (k)")
123     plt.ylabel("J(x)")
124     plt.grid(True)
125
126     plt.subplot(1, 3, 2)
127     plt.plot(k_range, res['iter_hist'])
128     plt.title("ADMM Iterations per MM Step")
129     plt.xlabel("Outer Iter (k)")
130     plt.ylabel("Inner Iters")
131     plt.grid(True)
132
133     plt.subplot(1, 3, 3)
134     plt.plot(k_range, res['err_hist'])
135     plt.title("Relative Error vs Outer Iter")
136     plt.xlabel("Outer Iter (k)")
137     plt.yscale('log')
138     plt.ylabel("||x_{k+1} - x_k|| / ||x_k||")
139     plt.grid(True)
140
141     plt.show()
142
143 # 3. Lambda Sensitivity Plot (New)
144 plt.figure(figsize=(10, 6))
145 for i, p in enumerate(p_values):
146     res = results_by_p[i]
147     plt.semilogx(
148         res["lambda_list_for_plot"],
149         res["psnr_list_for_plot"],
150         'o-',
151         linewidth=2,
152         label=f'p={p}'
153     )

```

```

153
154 plt.title("Parameter Sensitivity: PSNR vs  $\lambda$ ")
155 plt.xlabel(" $\lambda$  (log scale)")
156 plt.ylabel("PSNR (dB)")
157 plt.legend()
158 plt.grid(True, which="both", ls="-")
159 plt.tight_layout()
160 plt.show()
161
162 return [res['image'] for res in results_by_p]

```

A.2.2 Other Functions

This section contains those functions like psnr, sampling mask etc.

```

1 def dct2(x_flat, shape):
2     """
3     Performs 2D Orthonormal DCT on a flattened vector.
4     Reshapes to 2D -> Applies DCT -> Flattens back.
5     """
6     x = x_flat.reshape(shape)
7     return dct(dct(x.T, norm='ortho').T, norm='ortho').flatten()
8
9 def idct2(x_flat, shape):
10    """
11    Performs 2D Orthonormal IDCT on a flattened vector.
12    """
13    x = x_flat.reshape(shape)
14    return idct(idct(x.T, norm='ortho').T, norm='ortho').flatten()
15
16 def psnr(img, img_ref):
17     img = img.flatten()
18     img_ref = img_ref.flatten()
19     num = 1.0
20     if np.max(img_ref) > 1.1: num = 255.0
21     den = np.linalg.norm(img_ref - img) / np.sqrt(img_ref.shape
22 [0])
23     if den == 0: return 100
24     return 20 * np.log10(num / den)
25
26 def relative_change(x, y):
27     x = x.flatten()
28     y = y.flatten()
29     if np.linalg.norm(y) == 0: return 0
30     return np.linalg.norm(x - y) / np.linalg.norm(y)
31
32 def sampling_mask(N, idx):
33     mask = np.zeros((N,))
34     mask[idx] = 1

```

```

34     return mask
35
36 def Wtranspose(N, x, idx):
37     out = np.zeros((N,))
38     out[idx] = x
39     return out
40
41 def objective_function(idx, x, m, lammbda, p, img_shape, epsilon=1
42 e-6):
43     diff = x[idx] - m
44     term1 = np.sum(diff**2)
45     y = dct2(x, img_shape)
46     term2 = lammbda * np.sum((epsilon + np.abs(y))**p)
47     return term1 + term2
48
49 def soft_threshold(z, threshold):
50     return np.sign(z) * np.maximum(np.abs(z) - threshold, 0)

```

After this, all the output is obtained by simply calling the `reconstruct` function with the image and the sampling factor in the `wrapper.ipynb` file and the output images are saved in the respective folder.